

Alarm & Alert Processing System

Portfolio Technical Design Document & Production Architecture Specification

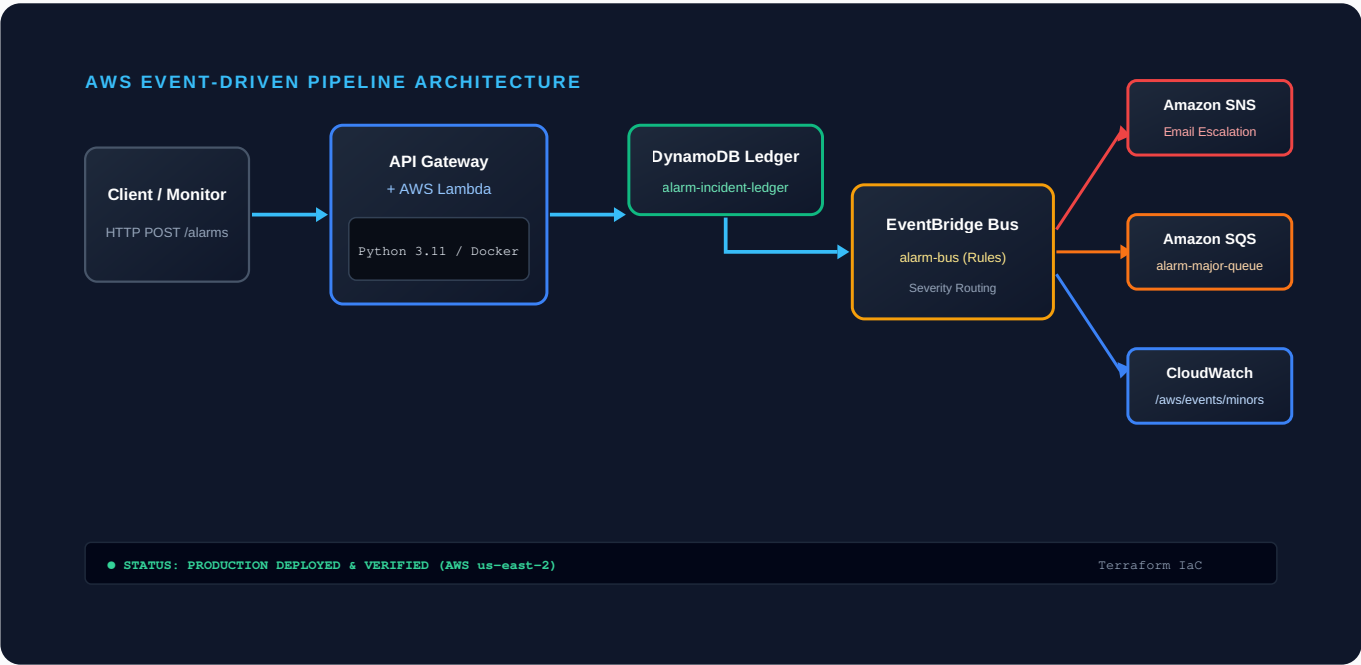
Author: DevOps & Platform Engineering

Environment: AWS Production (us-east-2)

Date: August 2026

1. Executive Summary & Architecture Overview

The **Alarm & Alert Processing System** is a high-performance, serverless event-driven architecture designed to ingest real-time system telemetry, evaluate severity thresholds, record immutable audit logs into DynamoDB, and fan out operational alerts across multiple dispatch channels. Built entirely on Amazon Web Services (AWS) using Terraform infrastructure-as-code, the platform guarantees sub-second processing latencies, robust fault tolerance, and secure containerized compilation for Python 3.11 serverless runtimes.



2. End-to-End System Flow & Architecture

Phase 1: Ingress & Compute (API Gateway & AWS Lambda)

Clients transmit JSON payloads via HTTP POST to API Gateway, which forwards requests using payload format version 2.0 to the core AWS Lambda function (`alarm_and_alert_processor`).

Phase 2: DynamoDB Ledger Persistence & State Lifecycle

The Lambda runtime performs an atomic write or update against the serverless **DynamoDB Ledger** (`alarm-incident-ledger`) using `alarm_id` as the partition key. It tracks active faults (`TRIGGERED`) and clear alarms (`RESOLVED`).

Phase 3: Event Bus Publishing & Fan-Out Routing

Events are published to a custom **EventBridge Bus** (`alarm-bus`) and routed based on severity: **CRITICAL** to SNS email escalation, **MAJOR** to SQS queue processing, and **MINOR** to CloudWatch logs.

3. DynamoDB Ledger Schema & Stored Item Structure

Attribute Name	Data Type	Key Type	Description
<code>alarm_id</code>	String (S)	Partition Key (PK)	Unique identifier for the alarm (e.g., <code>test-minor-001</code>)
<code>source</code>	String (S)	—	Originating telemetry or monitoring system component
<code>severity</code>	String (S)	—	Severity classification (<code>MINOR</code> , <code>MAJOR</code> , <code>CRITICAL</code>)
<code>state</code>	String (S)	—	Incident lifecycle state: <code>TRIGGERED</code> or <code>RESOLVED</code>
<code>message</code>	String (S)	—	Detailed status or event description message
<code>updated_at</code>	String (S)	—	ISO-8601 timestamp of last state transition

Stored Item Example (Resolved State)

```
{
  "alarm_id": { "S": "test-minor-001" },
  "source": { "S": "cpu_monitor" },
  "severity": { "S": "MINOR" },
  "state": { "S": "RESOLVED" },
  "message": { "S": "CPU utilization returned to normal baseline." },
  "updated_at": { "S": "2026-07-29T21:45:10Z" }
}
```

4. Key Engineering Solution: Containerized Lambda Build

Solution Implemented: To solve local host vs. Amazon Linux binary mismatches (`Runtime.ImportModuleError`), Terraform utilizes an automated Docker containerized build pipeline with

`null_resource` and official base image `public.ecr.aws/lambda/python:3.11` to compile precise wheels prior to packaging.

5. Complete Post-Deployment Testing & Verification Guide

Execute these commands against your deployed API Gateway endpoint to verify the full telemetry, storage, and fan-out pipeline.

Test 1: Minor Severity Alarm & Resolution

```
# Ingest Triggered Alarm
curl -X POST "YOUR_API_GATEWAY_ENDPOINT/alarms" -H "Content-Type: application/json" -d
'{"alarm_id": "test-minor-001", "source": "cpu_monitor", "severity": "MINOR", "state":
"TRIGGERED", "message": "CPU spike."}'

# Resolve Alarm
curl -X POST "YOUR_API_GATEWAY_ENDPOINT/alarms" -H "Content-Type: application/json" -d
'{"alarm_id": "test-minor-001", "source": "cpu_monitor", "severity": "MINOR", "state":
"RESOLVED", "message": "CPU normal."}'
```

Test 2: Major Severity Alarm & SQS Queue Verification

```
curl -X POST "YOUR_API_GATEWAY_ENDPOINT/alarms" -H "Content-Type: application/json" -d
'{"alarm_id": "test-major-002", "source": "api_gateway_latency", "severity": "MAJOR", "state":
"TRIGGERED", "message": "P99 latency high."}'

# Poll SQS Queue
aws sqs receive-message --queue-url "https://sqs.us-east-2.amazonaws.com/063815476191/alarm-
major-queue" --max-number-of-messages 5
```

Test 3: Critical Severity Alert & SNS Email Escalation

```
curl -X POST "YOUR_API_GATEWAY_ENDPOINT/alarms" -H "Content-Type: application/json" -d
'{"alarm_id": "prod-db-fail-01", "source": "database_cluster", "severity": "CRITICAL", "state":
"TRIGGERED", "message": "Primary DB unreachable."}'
```